

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 2 – SCENARIO BASED

Course Code:	DSA03	Course Title:	Natural Language Processing
CO Assessed:	CO2 – Manipulation	Assessment:	Assessment Tool 2 – SCENARIO BASED
Weightage:	35% of CIA		
CO5 Statement:	Design inflectional, derivational, finite-state parsing, stemming algorithms and for analyse morphological methods. (BL-3)		
Student Name:	M.Rishitha	Reg. No.:	192424335
Section / Batch:	2024 AIDS	Date:	06-08-2026

Code of Conduct

I M.Rishitha[192424335] (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: M.Rishitha

Instructions

- Read all questions carefully before attempting the answers.
- Answer all five questions. Each question carries equal marks.
- Show all intermediate steps, calculations, probability computations, tagging decisions, and justifications wherever applicable.
- Duration: 30 minutes.

Section / Topic	Focus	Q Nos
English Word Classes, Tagsets for English, Stochastic Part of Speech Tagging, HMM	Morphological parsing of disagree, agreement, and agreeable; identification of prefixes, suffixes, base forms, and semantic effects of derivation.	Q1

Annexure A – SCENARIO BASED

A company is developing an NLP-based grammar checker for educational applications. The system is trained using a manually POS-tagged corpus. Your task is to recreate the Hidden Markov Model (HMM) **without using any built-in NLP or HMM libraries**.

From the given corpus,

- determine the **Emission Probability**
- determine the **Transition Probability**
- implement the **Viterbi Algorithm**
- predict the POS tags for the new input sentence.

Use only Python dictionaries, loops, and basic programming constructs.

Training Corpus

Sentence	POS Tags
The/DT boy/NN eats/VBZ rice/NN	DT NN VBZ NN
The/DT girl/NN drinks/VBZ milk/NN	DT NN VBZ NN
A/DT cat/NN drinks/VBZ milk/NN	DT NN VBZ NN
The/DT dog/NN chases/VBZ cat/NN	DT NN VBZ NN
A/DT teacher/NN teaches/VBZ students/NNS	DT NN VBZ NNS
Students/NNS study/VBP English/NN	NNS VBP NN
Birds/NNS fly/VBP high/RB	NNS VBP RB
Children/NNS play/VBP games/NNS	NNS VBP NNS

New Input Sentence

The cat drinks milk

Tasks

1. Separate each sentence into words and POS tags.
2. Compute the **Emission Probability** of every word.
3. Compute the **Transition Probability** between POS tags.
4. Recreate the Hidden Markov Model using Python dictionaries.
5. Implement the **Viterbi Algorithm** without using any built-in HMM/NLP libraries.
6. Predict the POS tag sequence for "The cat drinks milk"

```
Python 3.10.11 (tags/v3.10.11:7d4cc5a, Apr 5 2023, 00:38:17) [MSC v.1929 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
===== RESTART: C:/Users/rishi/Downloads/ass.py =====
STEP 1 : Separate Words and POS Tags
=====

Sentence : The/DT boy/NN eats/VBZ rice/NN
Words : ['The', 'boy', 'eats', 'rice']
Tags : ['DT', 'NN', 'VBZ', 'NN']

Sentence : The/DT girl/NN drinks/VBZ milk/NN
Words : ['The', 'girl', 'drinks', 'milk']
Tags : ['DT', 'NN', 'VBZ', 'NN']

Sentence : A/DT cat/NN drinks/VBZ milk/NN
Words : ['A', 'cat', 'drinks', 'milk']
Tags : ['DT', 'NN', 'VBZ', 'NN']

Sentence : The/DT dog/NN chases/VBZ cat/NN
Words : ['The', 'dog', 'chases', 'cat']
Tags : ['DT', 'NN', 'VBZ', 'NN']

Sentence : A/DT teacher/NN teaches/VBZ students/NNS
Words : ['A', 'teacher', 'teaches', 'students']
Tags : ['DT', 'NN', 'VBZ', 'NNS']

Sentence : Students/NNS study/VBP English/NN
Words : ['Students', 'study', 'English']
Tags : ['NNS', 'VBP', 'NN']

Sentence : Birds/NNS fly/VBP high/RB
Words : ['Birds', 'fly', 'high']
Tags : ['NNS', 'VBP', 'RB']

Sentence : Children/NNS play/VBP games/NNS
Words : ['Children', 'play', 'games']
Tags : ['NNS', 'VBP', 'NNS']

=====
STEP 2 : Emission Probability
=====

TAG : DT
The : 0.6
A : 0.4

TAG : NN
boy : 0.1
rice : 0.1
girl : 0.1
milk : 0.2
cat : 0.2
dog : 0.1
teacher : 0.1
English : 0.1

TAG : VBZ
eats : 0.2
drinks : 0.4
chases : 0.2
teaches : 0.2

TAG : NNS
students : 0.2
Students : 0.2
Birds : 0.2
Children : 0.2
games : 0.2

TAG : VBP
study : 0.333
fly : 0.333
play : 0.333

TAG : RB
high : 1.0

=====
STEP 3 : Transition Probability
=====

corpus = [
    "The/DT boy/NN eats/VBZ rice/NN",
    "The/DT girl/NN drinks/VBZ milk/NN",
    "A/DT cat/NN drinks/VBZ milk/NN",
    "The/DT dog/NN chases/VBZ cat/NN",
    "A/DT teacher/NN teaches/VBZ students/NNS",
    "Students/NNS study/VBP English/NN",
    "Birds/NNS fly/VBP high/RB",
    "Children/NNS play/VBP games/NNS"
]

print("=="*60)
print("STEP 1 : Separate Words and POS Tags")
print("=="*60)
word_lists = []
tag_lists = []
for sentence in corpus:
    words = []
    tags = []
    pairs = sentence.split()
    for pair in pairs:
        word, tag = pair.split("/")
        words.append(word)
        tags.append(tag)
    word_lists.append(words)
    tag_lists.append(tags)

print("\nSentence :", sentence)
print("Words :", words)
print("Tags :", tags)

print("\n")
print("=="*60)
print("STEP 2 : Emission Probability")
print("=="*60)

tag_count = {}
emission_count = {}

for words, tags in zip(word_lists, tag_lists):
    for word, tag in zip(words, tags):
        tag_count[tag] = tag_count.get(tag, 0) + 1
        emission_count[tag] = emission_count.get(tag, {})

        for word in emission_count[tag]:
            probability = emission_count[tag][word] / tag_count[tag]
            emission_probability[tag][word] = probability

for tag in emission_probability:
    print("\nTAG :", tag)
    for word in emission_probability[tag]:
        print(word, ":", round(emission_probability[tag][word], 3))

print("\n")
print("=="*60)
print("STEP 3 : Transition Probability")
print("=="*60)

transition_count = {}
previous_count = {}

for tags in tag_lists:
    for i in range(len(tags)-1):
        previous = tags[i]
        current = tags[i+1]

        previous_count[previous] = previous_count.get(previous, 0) + 1

        if previous not in transition_count:
            transition_count[previous] = {}

        transition_count[previous][current] = transition_count[previous].get(current, 0) + 1

transition_probability = {}

for previous in transition_count:
    transition_probability[previous] = {}
```

```
=====
STEP 3 : Transition Probability
=====

DT
-> NN : 1.0

NN
-> VBE : 1.0

VBE
-> NN : 0.8
-> NNS : 0.2

NNS
-> VBP : 1.0

VBP
-> NN : 0.333
-> RB : 0.333
-> NNS : 0.333

=====
STEP 4 : HMM Created
=====

Emission Dictionary
('DT': ('The': 0.6, 'A': 0.4), 'NN': ('boy': 0.1, 'rice': 0.1, 'girl': 0.1, 'milk': 0.2,
'cat': 0.2, 'dog': 0.1, 'teacher': 0.1, 'English': 0.1), 'VBE': ('eats': 0.2, 'drinks':
0.4, 'chases': 0.2, 'teaches': 0.2), 'NNS': ('students': 0.2, 'students': 0.2, 'Birds':
0.2, 'Children': 0.2, 'games': 0.2), 'VBP': ('study': 0.3333333333333333, 'fly': 0.33333
3333333333, 'play': 0.3333333333333333), 'RB': ('high': 1.0))

Transition Dictionary
('DT': ('NN': 1.0), 'NN': ('VBE': 1.0), 'VBE': ('NN': 0.8, 'NNS': 0.2), 'NNS': ('VBP': 1
.0), 'VBP': ('NN': 0.3333333333333333, 'RB': 0.3333333333333333, 'NNS': 0.33333333333333
33))

=====
STEP 5 : Viterbi Prediction
=====

=====

previous_tag = None
for word in words:
    best_tag = None
    best_score = -1

    for tag in emission_probability:
        emission = emission_probability[tag].get(word,0)

        if previous_tag is None:
            score = emission
        else:
            transition = transition_probability.get(previous_tag, {}).get(tag,0)
            score = emission * transition

        if score > best_score:
            best_score = score
            best_tag = tag

predicted_tags.append(best_tag)
previous_tag = best_tag

return predicted_tags

sentence = "The cat drinks milk"
tags = viterbi(sentence)

print("\nInput Sentence")
print(sentence)

print("\nPredicted POS Tags\n")

for word, tag in zip(sentence.split(), tags):
    print(word, " ----> ", tag)
```

Rubrics for Calculation

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Scenario	20%	Demonstrates clear and complete understanding of the scenario and context	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Misinterprets or fails to understand the scenario
Identification of Key Issues	20%	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points	Unable to identify key issues
Application of Concepts	25%	Applies appropriate concepts effectively	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts

		to address the scenario			
Decision Making	20%	Makes well-reasoned, logical decisions with strong rationale	Makes reasonable decisions with some justification	Makes weak decisions with limited justification	No clear decisions made or rationale provided
Clarity of Response	15%	Response is clear, structured, and logically presented	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand

Q. No. / Criteria Level	Understanding of Scenario	Identification of Key Issues	Application of Concepts	Decision Making	Clarity of Response	Sub. Total	Total %
1	20	15	18	20	15	88	88
2							
3							
4							
5							

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____